# Energy Metering with the Portenta C33

This application note describes how to implement a simple energy meter with the Portenta C33.

Author José Bagur and Taddy Chung

Last revision 10/10/2024



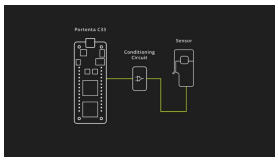
## Introduction

This application note explores the implementation of a simple yet useful energy meter using the Arduino® Portenta C33 and a non-invasive current transformer. The proposed energy meter enables real-time measurement and monitoring of electrical current consumption, providing valuable and useful insights into energy usage patterns to the user.

Non-invasive current transformers offer several advantages, including electrical safety, easy installation, and the ability to measure current in existing electrical circuits without interrupting the flow of current. These characteristics make them well-suited for

metering, power monitoring, and load management.

The Portenta C33 features a powerful microcontroller and onboard wireless connectivity, making it an ideal choice for energy monitoring applications. The Portenta C33's onboard Wi-Fi® module enables seamless integration with wireless networks and facilitates communication with the **Arduino Cloud platform**.




Portenta C33 with a non-invasive current transformer

By combining the Portenta C33 and the SCT013-000 current transformer, you can quickly build an energy meter that can measure Root Means Square (RMS) current, power consumption, and communicates the data to the Arduino Cloud platform for further analysis and visualization.

## Goals

The main goals of this application note are as follows:

- Develop a functional energy meter that can provide real-time insights into energy usage

track and analyze their energy consumption.

Showcase the integration of the Portenta C33 board with a non-invasive current transformer to measure AC current.

Calculate the RMS (Root Mean Square) value of a current waveform, providing an accurate representation of the actual current flowing through the circuit.

Use the measured RMS current and a known AC voltage to calculate power consumption in Watts.

Establish a connection between the Portenta C33 and the Arduino Cloud to send the measured RMS current data for further analysis, visualization, and remote monitoring.

# Hardware and Software Requirements

## Hardware Requirements

- **Portenta C33** (x1)
- USB-C® cable (x1)
- Wi-Fi® W.FL antenna (x1)
- SCT013-000 current transformer (x1)
- **Conditioner circuit** (x1)

## Software Requirements

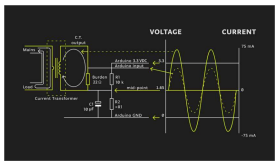
- **Arduino IDE 1.8.10+**,
- **Arduino IDE 2.0+**, or

For the Wi-Fi® connectivity feature of Portenta C33, we will use **Arduino Cloud**. In case you do not have an account, create one for free **here**.

The **energy meter example sketch**

## Hardware Setup Overview

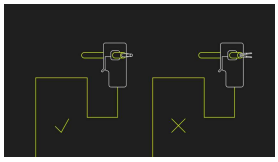
The electrical connections of the intended application design are shown in the diagram below:




Application's electrical connection overview

The overview illustrates the connection from the sensor passing through a conditioner circuit and interfacing with the Portenta C33.

**Take into account that only one AC wire must pass in between the current transformer**, as shown below:



Correct usage of a non-invasive current transformer

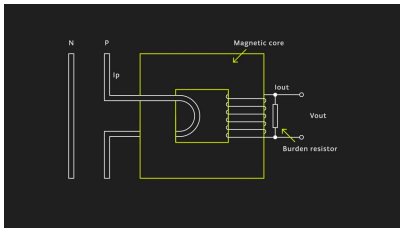
# Measuring Current Using Non-Invasive Current Transformers

Non-invasive current transformers, such as the SCT013-000 current transformer used in this application note, provide a safe and convenient method for measuring electrical current, without the need for direct electrical connections. These transformers work on the principle of **electromagnetic induction** and consist of a primary winding and a secondary winding.

As shown in the animation below, the primary winding typically consists of a single turn or a few turns of a conductor, while the secondary winding consists of a large number of turns wound around a magnetic core. When an alternating current flows through the primary winding, it induces a proportional current in the secondary winding. This induced current can be measured and used to determine the magnitude of the primary current.

As shown in the animation below, the primary winding is typically a single turn or a few turns of a conductor, while the secondary winding is a large number of turns wound around

alternating current flows through the primary winding, it induces a proportional current in the secondary winding. This induced current can be measured and used to determine the magnitude of the primary current.




To accurately measure the induced current, a **burden resistor** is connected in parallel with the secondary winding of the current transformer. The burden resistor creates a voltage drop proportional to the secondary current, which can be measured using an analog or digital sensor, for example, the analog input pins of the Portenta C33. In this application note, the measured secondary current is converted into an RMS current value using appropriate calibration ratios and calculations.

> The value of the burden resistor must be carefully chosen to match the physical characteristics of your current transformer and measurement sensor.

To calculate the ideal burden resistor some math is required. Below you can find the calculations done for the

Primary winding peak current ($I_P$):

$$I_P = I_{RMS} \times \sqrt{2} = (100\,A)(1.414) = 141.4\,A$$

Primary winding peak current calculation

Secondary winding peak current ($I_S$):

$$I_S = \frac{I_P}{N_{Turns}} = \frac{141.4\,A}{2000} = 0.0707\,A$$

Secondary winding peak current calculation

Ideal burden resistor ($R_{Burden}$):

$$R_{Burden} = \frac{A_{REF}/2}{I_S} = \frac{(3.1\,V/2)}{0.0707\,A} = 21.9\,\Omega$$

Ideal burden resistor calculation

For this application note and the used non-invasive current transformer, the ideal burden resistor $R_{Burden}$ is 21.9 Ω; you can use a commercially available resistor value, for example, 22 Ω.

Non-invasive current transformers offer several advantages, including electrical safety, easy installation, and the ability to measure current in existing electrical circuits without interrupting the current flow. These characteristics make them well-suited for applications such as:

- Energy metering
- Power monitoring

# Non-Invasive Current Transformer Example Sketch

The non-invasive current transformer sketch has a simple structure which is explained below.

```
/**
  Energy metering a
  Name: current_clo
  Purpose: This ske
  to measure RMS cu

  @author Arduino T
  @version 1.0 20/0
*/

// Import the prope
#include "thingProp

// Define a floatin
float   Sensor_Fact

// Define floating-
float   Irms, AP;

// Define an intege
int     Region_Volt

// Define a boolean
bool    Sample_Swit

// Define the pin w
#define TRANSFORMER
```

The following sections will help you to understand the main parts of the code.

## Variables and Constants

The key variables and constants used in the sketch are:

```
// Conversion factor
float Sensor_Factor =

// RMS Current and Ap
float Irms, AP;

// Voltage specific t
int Region_Voltage =

// Boolean variable t
bool Sample_Switch =

// Analog pin to whic
#define TRANSFORMER_P
```

`Sensor_Factor`: A conversion factor used to convert the raw current transformer reading to an RMS current value.

`Irms` and `AP`: Variables used to store the RMS current and apparent power respectively.

`Region_Voltage`: The voltage of the power system you are monitoring. This varies depending on the country and the type of power system.

`Sample_Switch`: A boolean switch to control whether or not your Portenta C33 should be measuring power. If it is true, your Portenta C33 will measure power.

`TRANSFORMER_PIN`: The pin on your Portenta C33 board that the current transformer is connected

## Initialization Function

The `setup()` function helps set up the communication with the Arduino Cloud, as well as the communication settings of the board itself:

```
void setup() {
  // Initialize the s
  Serial.begin(115200
  // Set the current
  // Set the resoluti
  // Pause the execut
  pinMode(TRANSFORMER
  analogReadResolutio
  delay(1000);

  // Call the functio
  iot_cloud_setup();
}
```

This function runs once when the Portenta C33 starts:

- It initializes the serial communication.
- Sets the pin mode of the current transformer input pin to input.
- Sets the ADC resolution to 12 bits and waits for a second for the system to stabilize.
- Finally calls the `iot_cloud_setup()` function to set up the Arduino Cloud connection.

## The Main Loop

The main loop of the sketch is as follows:

```
void loop() {
  // Update the stat
  ArduinoCloud.updat

  // If the sampling
  if (Sample_Switch
    Irms = getCurrer

    // Calculate app
    AP = Irms * Regi

    // Print RMS cur
    Serial.print(F("
    Serial.print(Irm
    Serial.print(F("
    Serial.print(AP)
  } else {
    // If the sampli
    Serial.println(F
  }

  // Update the RMS
  cloud_Current = Ir
  cloud_ApparentPowe

  // Pause the execu
  delay(1000);
}
```

The main `loop()` function executes continuously. At each iteration:

- It first updates the Arduino Cloud connection.
- If the `Sample_Switch` is true, it measures current, calculates apparent power, and prints them to the IDE's Serial Monitor.
- If `Sample_Switch` is false, it simply prints that the energy measurement is paused.
- It then updates the `cloud_Current` and `cloud_ApparentPower` variables with the local values of current and apparent power, then

## User Functions

The `iot_cloud_setup()` function groups the initial configuration process of the Arduino Cloud as follows:

```
void iot_cloud_setup(
  // Defined in thing
  initProperties();

  // Connect to Ardui
  ArduinoCloud.begin(

  /*
     The following fu
     related to the s
     the higher numbe
     The default is 0
     Maximum is 4
 */
  setDebugMessageLeve
  ArduinoCloud.printD

  sct_ratio = Sensor_
  system_Voltage = Re
  sample_Control = Sa
}
```

This function is responsible for setting up the connection with the Arduino Cloud:

- It initiates the properties of the Cloud connection.
- Begins the Cloud connection with the preferred connection method.
- Sets the debug message level to 2 for detailed debugging, and prints the debug information.

The variables `sct_ratio`,

synchronize the local values with the Arduino Cloud.

Now, let's talk about the `getCurrent()` function:

```
float getCurrent() {
  float Current = 0;
  float I_Sum = 0;
  int N = 0;
  long time = millis

  // Collect samples
  while(millis() - t

    // Read the anal
    int sensorValue

    // Convert the a
    float sensorVolt

    // Convert the s
    Current = sensor

    // Calculate the
    I_Sum += sq(Curr
    N++;
    delay(1);
  }

  // Compensate for
  I_Sum = I_Sum * 2;

  // Calculate RMS c
```

The `getCurrent()` function calculates the RMS current from the sensor reading. It reads the sensor value, converts it to voltage, then calculates the current. The square of the current is summed over 0.5 seconds (approximately 30 cycles at 60 Hz). This sum is compensated for the negative semi-cycle quadratics and then

current. This value is returned to the user.

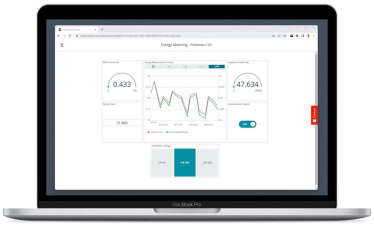
Finally, `onSctRatioChange()`, `onSystemVoltageChange()`, and `onSampleControlChange()` functions: These functions get executed every time the corresponding value is changed from the Arduino Cloud. For example, if `onSctRatioChange()` is executed, the `Sensor_Factor` will be updated with the new value received from the Cloud, and similarly for the others.

## Full Example Code

The complete example code can be downloaded here. The `thingProperties.h` header is already included for your reference, and it is based on the variables of the example code. The header is generated automatically with Arduino Cloud. If you desire to modify the requirements of the application environment, it is recommended to make changes within the Arduino Cloud environment.

## Arduino Cloud Dashboard

The Arduino Cloud allows to create a dashboard with professional real-time Human-Computer Interaction (HCI) as can be seen in the following animation, showing an active



One of the standout features of the Arduino Cloud Dashboard is the ability to update the current transformer's ratio in real time. This feature becomes particularly useful when you need to switch to a different current transformer in a live deployment scenario. Different current transformers can possess different electrical characteristics more or less convenient for different scenarios, and the real-time update capability simplifies the swap between them.

Additionally, the dashboard lets you select the installation voltage to meet the requirements of your site. This adaptability underscores the flexibility and user-friendliness of the Arduino Cloud Dashboard, making it an invaluable tool for handling real-time sensor data.

On a mobile phone, the Arduino Cloud dashboard displays the information as the previous figure. It provides the complete interface you would use on a desktop platform, making it a very useful feature to access the dashboard wherever you are.

## Conclusions

In this application note, we delved into the interaction between a Portenta C33 board and the SCT013-000 current transformer. We examined how the Arduino Cloud enables us to visualize and analyze real-time and historical sensor data intuitively.

By using the Portenta C33 board and the Arduino Cloud, you can transform raw sensor data into meaningful insights. Whether it is reading the RMS current and apparent power information, altering the current transformer configurations in real-time, or adapting to unique site requirements, this application note offers a robust and versatile system for handling these tasks.

One of the key takeaways from this application note is its potential for this application in various real-world scenarios. By integrating IoT and Cloud capabilities, we can effectively monitor and manage energy usage, leading to more efficient

contributing to a sustainable future.

## Next Steps

Now that you have learned to deploy a Portenta C33 with SCT013-000 current transformer, using the on-demand remote actuation and real-time data visualization of the Arduino Cloud platform, you will be able to expand the application further by adding new measurement equipment with similar characteristics. You could also deploy multiple sensors connected to different boards, creating a cluster to gather energy measurements from every point of interest in an electrical installation.